

1. Project type & language

Full-stack AI web application

Frontend

- React
- Vite
- JavaScript
- Tailwind CSS

Backend

- Python
- FastAPI

AI

- One LLM API
- LangGraph for the AI Agent
- RAG using embeddings + ChromaDB
- PDF/TXT document processing

Database

- SQLite for application metadata and investigation history

The application must contain exactly one primary AI Agent called **Knowledge Investigator Agent**.

Required AI architecture:

RAG → AI Agent → LLM → Knowledge Classification

The LLM, AI Agent, and RAG must be genuinely implemented, not simulated or hard-coded.

2. Main functionality / requirements

Build **KnowledgeGuard AI**, an AI knowledge reliability system.

The problem:

Modern AI systems depend on knowledge stored in documents and knowledge bases. This information can become outdated, contradictory, or uncertain over time.

The application should investigate knowledge and classify it as:

- CURRENT
- OUTDATED
- CONFLICTING
- UNCERTAIN

Required features

1. Upload PDF/TXT documents.
2. Extract and clean document text.
3. Split documents into chunks.
4. Generate embeddings.
5. Store embeddings in ChromaDB.
6. Store document metadata:
 - source
 - date
 - version
 - topic
7. Implement genuine RAG retrieval.
8. Implement one **Knowledge Investigator Agent** using LangGraph.
9. Agent should:
 - understand the claim
 - search the knowledge base
 - retrieve evidence
 - inspect metadata
 - determine whether more evidence is required
 - compare evidence
 - send structured evidence to the LLM
 - classify the knowledge
 - generate an explanation

- generate a recommendation

10. Use one LLM for meaningful reasoning.
11. Display retrieved evidence.
12. Display source/date/version.
13. Display reasoning.
14. Display classification.
15. Display confidence based on the implemented reasoning approach.
16. Display recommended action.
17. Display whether human verification is required.
18. Store investigation history.
19. Create a dashboard with dynamic statistics.
20. Create sample documents for:
 - CURRENT
 - OUTDATED
 - CONFLICTING
 - UNCERTAIN

Main pages

- Dashboard
- Knowledge Base
- Investigate
- Investigation Result
- Investigation History
- About

Example investigation

User enters:

"Is API v2 still recommended?"

Knowledge base contains:

Old document:

"API v2 is recommended."

New document:

"API v2 has been deprecated. API v3 is recommended."

The actual RAG + Agent + LLM workflow should identify this as:

OUTDATED

and show the evidence and reasoning.

Do not hard-code this result.

3. Project structure preferences

Use a clean full-stack structure:

knowledgeguard-ai/

```
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   ├── pages/
│   │   ├── services/
│   │   ├── hooks/
│   │   └── App.jsx
│   ├── package.json
│   └── ...
```

```
├── backend/
│   ├── api/
│   ├── agents/
│   ├── rag/
│   ├── llm/
│   ├── database/
│   ├── models/
│   ├── services/
│   ├── utils/
│   └── main.py
```

```
├── data/
│   └── sample_documents/
```

```
├── .env
├── .env.example
├── .gitignore
├── README.md
└── requirements.txt
```

Environment

Use a Python virtual environment for the backend.

Do not use Docker unless it becomes necessary.

Do not add CI/CD configuration because this is a 3-day academic MVP.

Keep the project easy to run locally.

4. Existing files

There are no existing project files that need to be preserved.

Create the project from scratch in the current workspace.

First inspect the workspace. If an existing project is already present, do not delete it blindly. Determine whether it belongs to this project before modifying anything.

You should create all required files and folders.

Important implementation priorities

Because this project must be completed within 3 days, prioritize:

Priority 1 — AI functionality

1. Document upload
2. Text extraction
3. Chunking
4. Embeddings
5. ChromaDB
6. RAG retrieval
7. LLM
8. Knowledge Investigator Agent

9. Classification

10. Evidence + reasoning

Priority 2 — Application

11. Dashboard

12. Knowledge Base

13. Investigation page

14. Investigation history

15. Error handling

Priority 3 — Polish

16. Responsive UI

17. Charts

18. Animations

19. Additional document formats

Do not sacrifice the RAG + Agent + LLM workflow for visual features.

Required architecture

The final application must clearly demonstrate:

User Claim

↓

RAG

Retrieves relevant evidence.

↓

Knowledge Investigator Agent

Investigates evidence and decides the next step.

↓

LLM

Reasons over the retrieved evidence.

↓

Classification

CURRENT / OUTDATED / CONFLICTING / UNCERTAIN

↓

Evidence + Explanation + Recommendation

↓

Human Verification

Important constraints

Do NOT build:

- generic chatbot
- simple PDF chatbot
- simple document summarizer
- fake AI Agent
- fake RAG
- hard-coded classifications
- multiple unnecessary agents
- multi-agent architecture
- fine-tuning
- complicated microservices
- unnecessary authentication
- mobile app
- blockchain

The goal is a reliable working MVP that clearly demonstrates **LLM + RAG + AI Agent**.

Build, run, test, and fix the application rather than only generating the source code.